

JAVA CONTROL STATEMENTS

A programming language uses control statements to cause the flow of execution to advance and branch based on changes to the state of a program. Java's program control statements can be put into the following categories: **selection, iteration, and jump**. Selection statements allow your program to choose different paths of execution based upon the outcome of an expression or the state of a variable. Iteration statements enable program execution to repeat one or more statements (that is, iteration statements form loops). Jump statements allow your program to execute in a nonlinear fashion.

Java supports two selection statements: if and switch. These statements allow you to control the flow of your program's execution based upon conditions known only during run time.

The switch statement is Java's multiway branch statement. It provides an easy way to dispatch execution to different parts of your code based on the value of an expression. As such, it often provides a better alternative than a large series of if-else-if statements.

The general form of a switch statement:

```
switch (expression) {  
    case value1:  
        // statement sequence  
        break;  
    case value2:  
        // statement sequence  
        break;  
    . . .  
    case valueN :  
        // statement sequence  
        break;  
    default:  
        // default statement sequence  
}
```

For versions of Java prior to JDK 7, **expression must be of type byte, short, int, char, or an enumeration**. Beginning with JDK 7, expressions can also be of **type String**. Each value specified in the case statements must be a unique constant expression (such as a literal value). Duplicate case values are not allowed. The type of each value must be compatible with the type of expression. The break statement is optional. If you omit the break, execution will continue on into the next case.

Java's iteration statements are for, while, and do-while. These statements create what we call loops. A loop repeatedly executes the same set of instructions until a termination condition is met. The do-while loop is especially useful when you process a menu selection, because you will usually want the body of a menu loop to execute at least once.

Beginning with JDK 5, there are two forms of the for loop. The first is the traditional form that has been in use since the original version of Java. The second is the newer “**for-each**” form.

```
for(initialization; condition; iteration) {  
    // body  
}
```

When you declare a variable inside a for loop, the scope of that variable ends when the for statement does. (That is, the scope of the variable is limited to the for loop.) Outside the for loop, the variable will cease to exist. If you need to use the loop control variable elsewhere in your program, you will not be able to declare it inside the for loop.

the **for** statement. The advantage of this approach is that no new keyword is required, and no preexisting code is broken. The for-each style of **for** is also referred to as the *enhanced for* loop.

The general form of the for-each version of the **for** is shown here:

```
for(type itr-var: collection) statement-block
```

The for-each style automates the preceding loop. Specifically, it eliminates the need to establish a loop counter, specify a starting and ending value, and manually index the array. Instead, it automatically cycles through the entire array, obtaining one element at a time.

The break statement has three uses. First, it terminates a statement sequence in a switch statement. Second, it can be used to exit a loop. Third, it can be used as a “civilized” form of goto. Java does not have a goto statement because it provides a way to branch in an arbitrary and unstructured manner. This usually makes goto-ridden code hard to understand and hard to maintain. It also prohibits certain compiler optimizations. There are, however, a few places where the goto is a valuable and legitimate construct for flow control. For example, the goto can be useful when you are exiting from a deeply nested set of loops. To handle such situations, Java defines an expanded form of the break statement. By using this form of break, you can, for example, break out of one or more blocks of code. These blocks need not be part of a loop or a switch. They can be any block. Further, you can specify precisely where execution will resume, because this form of break works with a label, and break gives you the benefits of a goto without its problems.

The general form of the labeled break statement:
break label;

The label is the name of a label that identifies a block of code. This can be a stand alone block of code but it can also be a block that is the target of another statement. When this form of break executes, control is transferred out of the named block. The labeled block must enclose the break statement, but it does not need to be the immediately enclosing block. This means, for example, that you can use a labeled break statement to exit from a set of nested blocks. But you cannot use break to transfer control out of a block that does not enclose the break statement. To name a block, put a label at the start of it. A label is any

valid Java identifier followed by a colon. Once you have labeled a block, you can then use this label as the target of a break statement. Doing so causes execution to resume at the end of the labeled block.

the following program shows three nested blocks, each with its own label. The **break** statement causes execution to jump forward, past the end of the block labeled **second**, skipping the two **println()** statements.

```
// Using break as a civilized form of goto.
class Break {
    public static void main(String args[]) {
        boolean t = true;

        first: {
            second: {
                third: {
                    System.out.println("Before the break.");
                    if(t) break second; // break out of second block
                    System.out.println("This won't execute");
                }
                System.out.println("This won't execute");
            }
            System.out.println("This is after second block.");
        }
    }
}
```

Running this program generates the following output:

```
Before the break.
This is after second block.
```

Sometimes it is useful to force an early iteration of a loop. That is, you might want to continue running the loop but stop processing the remainder of the code in its body for this particular iteration. The continue statement performs such an action. In while and do-while loops, a continue statement causes control to be transferred directly to the conditional expression that controls the loop. In a for loop, control goes first to the iteration portion of the for statement and then to the conditional expression. For all three loops, any intermediate code is bypassed.

The return statement is used to explicitly return from a method. That is, it causes program control to transfer back to the caller of the method. Thus, the return statement immediately terminates the method in which it is executed.

In addition to the jump statements, Java supports one other way that you can change your program's flow of execution: through exception handling. Exception handling provides a structured method by which run-time errors can be trapped and handled by your program. It is supported by the keywords try, catch, throw, throws, and finally. In essence, the exception handling mechanism allows your program to perform a nonlocal branch.